

Signa — Împărțirea muncii pe 5 persoane

Aplicație PWA „Duolingo pentru Limba Semnelor Române” (LSR). Recunoașterea rulează pe dispozitiv (MediaPipe + TensorFlow.js)
— fără cloud, fără costuri.

Starea actuală (iulie 2026)

- ☒ **Faza 1** — Cameră + 21 puncte MediaPipe + `normalize()`
- ☒ **Faza 2** — Colectare dataset (CollectPage: etichete libere, mod Foto/Video, import/export JSON)
- ☒ **Faza 3** — Antrenare în browser (TrainPage) + predicție live (model static: 25 litere, validat 250/250)
- ☒ **Faza 4** — Lecții gamificate (5 lecții × 5 litere, XP + stele)
- ☒ **Faza 4.5** — Semne dinamice (J, Z, X, Î, Ș, Ț) — pipeline GRU există, datele trebuie colectate
- ☐ **Extindere** — Cuvinte întregi (colectarea permite deja etichete libere)
- ☐ **Faza 5** — Backend (Supabase), conturi, clasament

Reguli de aur (valabile pentru toți)

1. `src/utils/normalize.js` **NU se modifică**. Orice schimbare invalidează modelul și tot dataset-ul.
2. Recunoașterea rămâne **pe dispozitiv** — niciodată imagini/video în cloud.
3. Stil: mobile-first, dark theme (`slate-900`), accent verde (`signa-400 = #34d399`).
4. Dataset-ul se exportă des (localStorage se poate umple) și se păstrează versiuni în `~/Downloads` sau un folder partajat.

Persoana 1 — Lead AI & Model (coordonator ML)

Responsabil de: clasificatoare, pipeline-ul dinamic, calitatea predicției. Deține regulile despre `normalize()` și arhitectura modelelor.

Fișiere principale: `src/hooks/useClassifier.js`, `src/pages/TrainPage.jsx`, `src/utils/normalize.js` (doar pază, nu modificare), `public/models/`

To do

- [] Antrenează și validează modelul dinamic (GRU) cu datele colectate pentru J, Z, X, Î, Ș, Ț
- [] Definește pragurile de încredere pentru predicția dinamică (confidence + margin) și integrează-le în CameraPage
- [] Decide strategia pentru cuvinte: model separat pentru cuvinte-semn vs. dactilare literă-cu-literă
- [] Adaugă evaluare pe set de test separat (nu doar validationSplit) în TrainPage
- [] Documentează procedura de reantrenare (pas cu pas, inclusiv capcanele cu redenumirea fișierelor și cache-ul HMR)
- [] Verifică performanța pe telefoane mai slabe (FPS predicție, memorie TF.js)

Persoana 2 — Date & Colectare (dataset owner)

Responsabil de: calitatea și acoperirea dataset-ului. Ideal cineva care cunoaște LSR sau lucrează cu un vorbitor nativ.

Fișiere principale: `src/pages/CollectPage.jsx`, `src/hooks/useDatasetCollector.js`, `src/data/lsr-alphabet.js`

To do

- [] Colectează secvențe video pentru literele dinamice: J, Z, X, Î, Ș, Ț (minim 30 înregistrări/literă, mai mulți semnatori)
- [] Stabilește lista primelor 20–30 de cuvinte LSR de colectat (salut, mulțumesc, familie, culori, cifre)
- [] Colectează cuvintele stabilite folosind câmpul de etichetă liberă + modul potrivit (Foto/Video)
- [] Diversifică datele: mâna stângă/dreaptă, unghiuri, distanțe, iluminare diferită
- [] Organizează fișierele exportate (convenție de nume, folder partajat, log cu cine/ce/când a colectat)
- [] Adaugă în CollectPage o vedere de ansamblu a dataset-ului (ce etichete există, câte exemple fiecare, ce lipsește)

Persoana 3 — Frontend & Gamificare (UX owner)

Responsabil de: experiența de învățare, lecții, design vizual, motivație (Duolingo-feel).

Fișiere principale: `src/pages/LessonsPage.jsx`, `src/pages/LessonPage.jsx`, `src/pages/HomePage.jsx`, `src/data/lessons.js`, `src/hooks/useProgress.js`, `src/components/lesson/`

To do

- [] Adaugă lecții pentru literele dinamice după ce modelul GRU e integrat
- [] Creează lecții pentru cuvinte (referință video/animație → imită → feedback → scor)
- [] Sistem de streak zilnic (zile consecutive de exercițiu) + notificări PWA
- [] Ecran de recapitulare/repetiție spațiată pentru literele deja învățate
- [] Animații de recompensă (confetti, level-up) și sunete de feedback
- [] Onboarding pentru utilizatori noi (permisiune cameră, cum ții mâna, primul semn ghidat)
- [] Audit de accesibilitate: contrast, dimensiuni touch target, funcționare fără sunet

Persoana 4 — Backend & Conturi (Faza 5)

Responsabil de: infrastructura Supabase, autentificare, sincronizare progres, clasament. **Atenție:** doar date de progres/profil pe server — niciodată imagini sau landmarks.

Fișiere noi: `src/lib/supabase.js`, hooks de sincronizare, pagini de profil/clasament

To do

- [] Configurează proiectul Supabase (auth cu email + Google, tabele: profiles, progress, leaderboard)
- [] Autentificare în aplicație (login/register/logout, sesiune persistentă offline-first)
- [] Sincronizarea progresului local (XP, stele, streak) cu serverul — localStorage rămâne sursa offline
- [] Pagina de clasament (săptămânal + all-time)
- [] Politici RLS (row-level security) pe toate tabelele
- [] Strategie de merge la conflict (progres local vs. server, ex. utilizator pe 2 dispozitive)
- [] Pagină de profil (nume, avatar, statistici)

Persoana 5 — QA, PWA & Livrare (release owner)

Responsabil de: calitate, testare pe dispozitive reale, experiența PWA, deploy.

Fișiere principale: `vite.config.js` (vite-plugin-pwa), manifest, service worker, CI/CD

To do

- [] Testare pe dispozitive reale: iPhone (Safari), Android (Chrome) — cameră, predicție, lecții
- [] Verifică instalarea PWA (add to home screen, splash, icoane, funcționare offline)
- [] Configurează deploy automat (Vercel/Netlify/Cloudflare Pages) cu preview per branch
- [] Adaugă teste automate pentru logica pură: `normalize()` (snapshot — să nu se schimbe!), validatori dataset, `useProgress`
- [] Pagină de diagnostic (versiune model, FPS, stare cameră) pentru debugging pe teren
- [] Gestionarea versiunilor de model (cum ajunge un model nou la utilizatori fără cache vechi)
- [] Testare cu utilizatori reali din comunitatea surzilor + colectare feedback structurat

Dependențe între persoane

```
P2 (date dinamice) → P1 (model GRU) → P3 (lecții litere dinamice)
P2 (date cuvinte) → P1 (model cuvinte) → P3 (lecții cuvinte)
P4 (backend) → P3 (clasament în UI)
P5 (deploy) ← toți (orice merge în producție trece prin P5)
```

Ordinea recomandată de start: P2 începe imediat colectarea (blochează totul), P1 pregătește pipeline-ul de evaluare, P3 lucrează în paralel la streak/onboarding (fără dependențe), P4 pornește Supabase de la zero, P5 pune deploy-ul și testele de la început.

Flux de lucru Git

Repozitoriu: <https://github.com/margi-tech/signa>

1. **main rămâne mereu funcțional** — nimeni nu face push direct pe el (protejat pe GitHub, orice schimbare trece printr-un Pull Request).
2. **Ramuri per funcționalitate, nu per persoană** — o ramură trăiește câteva zile, nu săptămâni. Numele începe cu rolul tău: `bash git checkout main && git pull git checkout -b p3/streak-zilnic` Exemple: `p1/model-gru`, `p2/colectare-cuvinte`, `p4/supabase-auth`, `p5/deploy-vercel`
3. **Când e gata, deschide un Pull Request** spre `main`: `bash git push -u origin p3/streak-zilnic gh pr create #` sau din interfața GitHub
4. **Altceva se uită pe cod înainte de merge** — minim o aprobare.
5. **După merge, toți se actualizează:** `git checkout main && git pull`.

Reguli speciale pentru PR-uri

- Un PR care atinge `normalize.js`, formatul dataset-ului sau `public/models/` se anunță întregii echipe și așteaptă acordul lui P1 (Lead AI).
- PR-uri mici și dese > PR-uri uriașe. Dacă ai depășit ~400 de linii modificate, probabil trebuia împărțit.
- Descrierea PR-ului spune **ce** și **de ce**, plus cum s-a testat (ideal cu un screenshot pentru schimbări de UI).
- Conflictul se rezolvă pe ramura ta (`git merge main`), nu pe `main`.

Ritm de lucru sugerat

- **Sync scurt săptămânal:** fiecare spune ce a terminat, ce urmează, ce îl blochează
- **Dataset-ul se exportă și se versionează la fiecare sesiune de colectare**

- Orice schimbare care atinge `normalize()` , formatul dataset-ului sau formatul modelului se anunță întregii echipe înainte de merge